Final Project Tutorial

Design of ECC Systems for Reliable Data Transmission

Tzu-Sian Wang

IIS5008 Hardware Security

Due: June 19, 2026, 23:59:59

Outline

1. Project Background
2. Flash Error Model
3. Program Interface
4. How to Write the Solution
5. Scoring
6. Testing and Submission
7. Design Strategy

Outline

1. Project Background
2. Flash Error Model
3. Program Interface
4. How to Write the Solution
5. Scoring
6. Testing and Submission
7. Design Strategy

Motivation

- Modern systems rely heavily on non-volatile memories such as NAND Flash.
- Real Flash errors are not only random bit flips.
- Errors may be structured, localized, time-dependent, and non-uniform.
- The goal is to design an ECC-based protection system that remains reliable under realistic Flash-inspired corruption.

Main Challenge

Balance three competing goals: exact recovery, low redundancy, and efficient runtime.

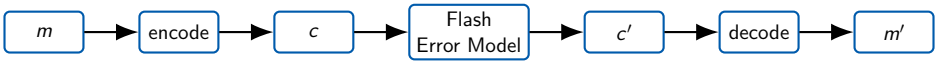
Error-Correcting Codes

- ECC adds structured redundancy to original data.
- The decoder uses redundancy to detect and correct corrupted bits or symbols.
- ECC is widely used in memory and storage systems to improve reliability.

In This Project

You need to design an ECC scheme for storage under a realistic NAND Flash error model.

System Flow



- m: original message
- c: encoded codeword
- c': corrupted codeword
- m': decoded output

Project Goal

Correctness Target

Design an ECC scheme such that

$$m' = m$$

under the evaluator's hidden Flash error conditions.

- Minimize redundancy.
- Keep runtime efficient.
- Avoid overfitting to one exact corruption pattern.

Outline

1. Project Background
2. Flash Error Model
3. Program Interface
4. How to Write the Solution
5. Scoring
6. Testing and Submission
7. Design Strategy

Channel Overview

The evaluator does not use a simple independent random bit-flip channel.

It may combine several Flash-inspired behaviors:

- random bit errors
- burst errors
- retention-related aging errors
- disturb-like clustered errors

Important

The exact corruption pattern of each hidden test case is not revealed in advance.

Random Bit Errors

- Sparse random bit flips across the encoded codeword.
- Simulates baseline noise, imperfect sensing, or mild instability.
- Usually low to moderate severity.
- May appear alone or together with other error types.

Design Implication

A baseline ECC layer should handle scattered independent errors efficiently.

Burst Errors

- Several consecutive bytes may be corrupted within a local region.
- Simulates localized failure or page-level instability.
- Challenging for schemes that assume all errors are independent.

Example

Original: A B C D E F G
Corrupted: A X X X E F G

Retention Errors and Aging

Each test case has an aging parameter T:

$$1 \leq T \leq 1000$$

- Larger T means the stored data is considered older.
- Older data is more vulnerable to retention-related corruption.
- Both encoder and decoder receive the same T.

Reference Retention Severity

The evaluator uses a reference severity function:

$$p(T) \in [(1-\epsilon)\hat{p}(T), (1+\epsilon)\hat{p}(T)]$$

$$\hat{p}(T) = 0.06\sqrt{\frac{T}{1000}}, \qquad \epsilon = 0.1$$

Example

If $T = 1000$, then $\hat{p}(1000) = 0.06$, so the actual severity may be in $[0.054, 0.066]$.

Disturb-like Cluster Errors

- Multiple bits or bytes may flip inside a small neighborhood.
- Simulates read/program disturb in dense Flash arrays.
- A cluster may overlap with random or burst errors.
- Unlike a burst, not every byte in the cluster must be corrupted.

Design Implication

Interleaving or block-level protection may help spread local damage.

Outline

1. Project Background
2. Flash Error Model
3. Program Interface
4. How to Write the Solution
5. Scoring
6. Testing and Submission
7. Design Strategy

Required Executable

Your submission must provide one executable named:

```
solution
```

It must support two modes:

```
./solution encode T <input_file> <output_file>
./solution decode T <input_file> <output_file>
```

- T is the storage aging parameter.
- You may adapt redundancy or layout based on T.

Encoder Input and Output

Encoder command:

```
./solution encode T <input_file> <output_file>
```

Input file format:

```
N
hex_string_of_length_2N
```

Example:

```
8
0123456789ABCDEF
```

Output: exactly one line containing the encoded hexadecimal codeword c.

Decoder Input and Output

Decoder command:

```
./solution decode T <input_file> <output_file>
```

Input file format:

```
N
corrupted_hex_codeword
```

Output: exactly one line containing a hexadecimal string of length $2N$.

Correct Output

The decoded message m' must be byte-by-byte identical to the original message m.

Output Rules

- Use only valid hexadecimal characters: 0–9 and A–F.
- Output length must be even.
- Do not write spaces, explanations, or debug messages.
- The program must not read or write files other than the specified input and output files.
- The solution must be deterministic.

Outline

1. Project Background
2. Flash Error Model
3. Program Interface
4. How to Write the Solution
5. Scoring
6. Testing and Submission
7. Design Strategy

Released Example Package

The file `exampleID_final.tar.gz` contains the public testing tools and public cases:

```
exampleID_final/
|-- evaluator
|-- grade.sh
|-- Makefile
`-- testcases/
    |-- public1/
    |-- public2/
    |-- public3/
    `-- public4/
```

Important

This package is for testing and reference. It includes tools and public cases, not a finished solution. Your real submission still needs your own `<StudentID>_final/` folder with source code and `bin/solution`.

Public Cases in the Package

Case	$T_{\max}$	Message Size	Baseline
public1	50	64 bytes	100%
public2	200	128 bytes	100%
public3	750	256 bytes	90%
public4	1000	512 bytes	60%

- Larger public cases have larger $T_{\max}$ and stronger aging stress.
- Passing public1 and public2 is the first checkpoint.
- public3 and public4 are better for testing robustness and trade-offs.

Recommended Work Plan

1. Build the command-line interface first.
2. Implement a very simple encode/decode pair and verify file I/O.
3. Add ECC protection block by block.
4. Add interleaving or stronger protection for larger T.
5. Run the public evaluator after every major change.
6. Record correctness, runtime, and redundancy for the report.

Development Rule

Make correctness stable first. Optimize redundancy and runtime after the decoder is reliable.

Project Folder to Create

```
<StudentID>_final/
|-- Makefile
|-- include/
|   |-- ecc.h
|   `-- utils.h
|-- src/
|   |-- main.cpp
|   |-- ecc.cpp
|   `-- utils.cpp
`-- bin/
    `-- solution
```

- `bin/solution` should be generated by `make`.
- Do not rely on files outside the official input and output paths.
- Keep testing tools outside your final submission folder unless the official instruction says otherwise.

Makefile Template

The example package uses a C++17-style Makefile:

```
CXX      ?= g++
CXXFLAGS ?= -O2 -std=c++17 -Wall -Wextra -Iinclude

SRC := src/main.cpp src/ecc.cpp src/utils.cpp
OBJ := $(SRC:src/%.cpp=build/%.o)
BIN := bin/solution

all: $(BIN)
$(BIN): $(OBJ) | bin
^^I$(CXX) $(CXXFLAGS) $(OBJ) -o $@
```

Check

If you use this template, create every file listed in `SRC`. From `<StudentID>_final/`, running `make` must create `./bin/solution`.

Main Program Skeleton

Your `main.cpp` should route the two modes:

```
int main(int argc, char** argv) {

    string mode = argv[1];
    int T = stoi(argv[2]);
    string input_path = argv[3];
    string output_path = argv[4];

    if (mode == "encode") {
        return run_encode(T, input_path, output_path);
    }
    if (mode == "decode") {
        return run_decode(T, input_path, output_path);
    }
    return 1;
}
```

File I/O Checklist

Encode input:

```
N
message_hex
```

Decode input:

```
N
corrupted_codeword_hex
```

- Parse N as the original message length in bytes.
- Convert the hex string to bytes before ECC processing.
- Write exactly one output line.
- Never print debug text into the output file.

ECC Design Starting Points

- Start with block-based protection so decoding stays simple.
- Add checksums or parity to detect uncorrectable blocks.
- Use interleaving to spread burst or cluster damage.
- Increase redundancy gradually as T becomes larger.
- Measure the encoded length every time: $|c_T| \leq 2|m|$.

Practical Mindset

Hidden tests may combine error types, so avoid a design that only handles one exact pattern.

Outline

1. Project Background
2. Flash Error Model
3. Program Interface
4. How to Write the Solution
5. Scoring
6. Testing and Submission
7. Design Strategy

Public Baselines

Testcase	Required Correctness
Public1	100%
Public2	100%
Public3	90%
Public4	60%

Hidden Case Score

Each hidden case is worth 10 points.

The score is divided into:

- Correctness: $50\% = 5$ points
- Runtime: $25\% = 2.5$ points
- Redundancy: $25\% = 2.5$ points

Key Idea

Correctness is the primary objective. Runtime and redundancy matter only if reliability is good enough.

Correctness Score

Correctness is the fraction of aging levels $T = 1, \ldots, T_{\text{max}}$ for which exact recovery succeeds.

$$S_{\text{corr}} = 2.5 + 2.5 \times \max\left(\frac{\text{self(corr)} - \text{avg(corr)}}{\max(\text{corr}) - \text{avg(corr)}}, -1\right)$$

- Score is clipped to $[0, 5]$.
- If the denominator is zero, all submissions receive 2.5 points.

Runtime and Redundancy

Runtime score:

$$S_{\text{run}} = 1.25 + 1.25 \times \max\left(\frac{\text{avg}(\text{runtime}) - \text{self}(\text{runtime})}{\text{avg}(\text{runtime}) - \min(\text{runtime})}, -1\right)$$

Redundancy ratio:

$$\frac{|c_T|}{|m|}$$

- Lower runtime is better.
- Lower average redundancy is better.

Correctness Gate and Hard Limits

If

$$\text{self(corr)} < \text{avg(corr)} - \sigma(\text{corr}),$$

then:

$$S_{\text{run}} = 0, \qquad S_{\text{red}} = 0.$$

Hard Constraints

- Maximum original input size: 512 bytes.
- Encoded length cap: $|c_T| \leq 2|m|$.
- Each encode/decode call has a 1-second time limit.

Outline

1. Project Background
2. Flash Error Model
3. Program Interface
4. How to Write the Solution
5. Scoring
6. Testing and Submission
7. Design Strategy

Unpack the Testing Package

First unpack the released example package:

```
tar -xzf exampleID_final.tar.gz
cd exampleID_final
```

The useful files are:

```
./evaluator
./grade.sh
./testcases/public1
./testcases/public2
./testcases/public3
./testcases/public4
```

Workflow

Keep your own solution folder next to this package, then call the evaluator with the path to your `bin/solution`.

Testcase Structure

```
testcases/
|-- public1/
|   |-- input.txt
|   `-- meta.txt
|-- public2/
|-- public3/
`-- public4/
```

Example input:

```
8
0123456789ABCDEF
```

Example meta:

```
id=example
T_max=5
msg_size=8
seed=3237998081
```

Build Your Solution First

From your own project folder:

```
cd <StudentID>_final
make clean
make
```

Check that the executable exists:

```
ls bin/solution
```

Expected Result

The evaluator will run `./bin/solution encode ...` and `./bin/solution decode ...`. If `bin/solution` is missing, the submission fails before correctness is tested.

Use Evaluator on One Case

Usage:

```
./evaluator <solution_path> <testcase_dir> [--quiet]
```

Example:

```
cd exampleID_final
./evaluator ../<StudentID>_final/bin/solution testcases/public1
```

Quieter output:

```
./evaluator ../<StudentID>_final/bin/solution testcases/public4 --quiet
```

`-quiet` Output Difference

The following shows the output shape. Actual numbers depend on the solution and testcase.

Without `-quiet`

```
$ ./evaluator ../114000000_final/bin/solution testcases/public1
T=1   PASS  time=1.42ms  |c|/|m|=1.31
T=2   PASS  time=1.37ms  |c|/|m|=1.31
T=3   FAIL  mismatch
...
T=50  PASS  time=1.40ms  |c|/|m|=1.31

Summary
N                : 64
T_max            : 50
correctness      : 49 / 50
total time       : 70.3 ms
avg |c|/|m|      : 1.3125
failures         : enc=0 dec=0 cap=0 mismatch=1 timeout=0
```

With `-quiet`

```
$ ./evaluator ../114000000_final/bin/solution testcases/public1
      -quiet
Summary
N                : 64
T_max            : 50
correctness      : 49 / 50
total time       : 70.3 ms
avg |c|/|m|      : 1.3125
failures         : enc=0 dec=0 cap=0 mismatch=1 timeout=0
```

Difference

`-quiet` suppresses the per-T pass/fail lines and keeps only the final summary.

What Evaluator Does

For every T in $1..T_{\max}$, the evaluator:

1. invokes encode to obtain c,
2. corrupts c using the Flash channel,
3. invokes decode to recover m',
4. compares m' with m and records metrics.

A Trial Fails If

The program crashes, exceeds time limit, outputs malformed hex, violates $|c| \leq 2|m|$, or reconstructs the wrong message.

Read the Evaluator Summary

The summary contains the information you need for debugging and the report:

```
N                 : original message size
T_max             : tested aging range
correctness       : correct_count / total_count
total time        : total encode/decode runtime
avg |c|/|m|       : average redundancy ratio
failures          : enc= dec= cap= mismatch= timeout=
```

- `mismatch`: decoder ran but recovered the wrong message.
- `cap`: encoded output violated $|c| \leq 2|m|$.
- `timeout`: at least one encode/decode call exceeded the limit.

Suggested Public Testing Order

```
./evaluator ../<StudentID>_final/bin/solution testcases/public1
./evaluator ../<StudentID>_final/bin/solution testcases/public2
./evaluator ../<StudentID>_final/bin/solution testcases/public3 --quiet
./evaluator ../<StudentID>_final/bin/solution testcases/public4 --quiet
```

- Start with public1 to catch interface and formatting bugs.
- Use public2 after basic ECC correction works.
- Use public3 and public4 to tune aging, burst, and cluster robustness.

Batch Test with grade.sh

After creating the final tarball:

```
mkdir self_test
cp <StudentID>_final.tar.gz self_test/
./grade.sh self_test ./testcases
cat grades.csv
```

- `grade.sh` extracts the tarball into a temporary directory.
- It removes old `bin/solution`, runs `make`, and calls the evaluator.
- It writes one CSV row per student and testcase.

Tip

Use direct `./evaluator` first when debugging one case; use `grade.sh` before final upload. If the script path layout differs locally, follow the official release layout or test cases one by one.

Report Contents

The report should include:

- names and student IDs,
- overview of the ECC system design,
- explanation of the main algorithms,
- handling of the Flash error model and aging parameter T,
- experimental results: correctness, runtime, redundancy,
- important implementation details,
- lessons learned and problems encountered,
- suggestions and feedback.

Submission Layout

```
<StudentID>_final/
|-- bin/
|   `-- solution
|-- src/
|   |-- main.cpp
|   `-- utils.cpp
|-- include/
|   `-- utils.h
|-- Makefile
`-- <StudentID>_report.pdf
```

Before evaluation:

```
make
./bin/solution ...
```

Compression

Compress the directory as:

```
tar -zcvf <StudentID>_final.tar.gz <StudentID>_final/
```

Example:

```
tar -zcvf 114062584_final.tar.gz 114062584_final/
```

Reminder

Incorrect folder structure or missing `bin/solution` may cause the program to be ungraded.

Final Grading

- Step 1: Public cases, 40%.
- Step 2: Hidden cases, 40%.
- Step 3: Report, 20%.

Bonus

Top 5 submissions for each hidden case receive +5 bonus points.
Each student can receive bonus from at most two hidden cases, up to +10 points.

Outline

1. Project Background
2. Flash Error Model
3. Program Interface
4. How to Write the Solution
5. Scoring
6. Testing and Submission
7. Design Strategy

Possible ECC Design Considerations

- Use stronger protection when T is large.
- Combine random-error correction with burst/cluster tolerance.
- Consider interleaving to spread localized errors across blocks.
- Keep metadata small because encoded length is capped at $2|m|$.
- Use deterministic decoding and avoid expensive search.

Trade-off Summary

Goal	Benefit	Risk
More redundancy	Better recovery	Lower redundancy score
Less redundancy	Better compression	More failures
Larger blocks	Lower overhead	Harder local correction
Smaller blocks	Localized recovery	More metadata overhead
T-adaptive design	Better aging response	More implementation complexity

Checklist

- `make` builds successfully from a clean directory.
- Executable is generated at `./bin/solution`.
- Encode/decode output contains exactly one valid hex line.
- Encoded length always satisfies $|c_T| \leq 2|m|$.
- Runtime stays below 1 second per call.
- Public cases are tested with the provided evaluator.
- Report explains design, error handling, results, and reflection.

Contact

Questions

For questions regarding the final project, post them on the EEClass discussion board first.

Thank you!